



Università degli Studi di Salerno

**Dipartimento di Ingegneria dell'Informazione ed Elettrica
e Matematica applicata**

Relazione Tecnica di Progettazione e Sviluppo

Progetto di Mobile Programming

MealMate

Meal Planner e Smart Pantry App

Tecnologie utilizzate: Dart • Flutter • SQLite -SharedPreferences

Gruppo 17:

*Ercolano Aniello
Cangianiello Francesco
Giannitti Serena
Rossomando Andrea*

Anno Accademico 2025/2026

Indice

1	Descrizione dell'applicazione	2
1.1	Obiettivo	2
1.2	Target Audience	2
1.3	Problemi risolti dall'applicazione	2
1.4	Principali scenari d'uso	3
2	Requisiti	5
2.1	Funzionalità implementate	5
2.2	Funzionalità considerate e non implementate	7
2.3	Feature avanzate scelte	7
2.4	Eventuali limitazioni note	7
3	Progettazione	8
3.1	Struttura generale	8
3.2	Schermate principali	9
3.3	Wireframe	10
3.4	Organizzazione dei Dati	14
4	Scelte tecnologiche	15
4.1	Framework utilizzato	15
4.2	Librerie principali	15
5	Implementazione	16
5.1	Widget principali	16
5.2	Organizzazione del codice	17
5.3	Gestione della Navigazione	17
5.4	Gestione dello Stato	18
5.5	Gestione della Persistenza	18
5.6	Parti di codice significative o complesse	19
6	Conclusione	20

1 Descrizione dell'applicazione

1.1 Obiettivo

“MealMate” è un'applicazione mobile sviluppata in Flutter che ha lo scopo di supportare l'utente nella gestione quotidiana della propria alimentazione.

L'app integra in un'unica piattaforma la pianificazione dei pasti e la gestione delle ricette, di una dispensa virtuale dove catalogare ingredienti e prodotti disponibili e di una lista della spesa.

L'app nasce dalla necessità di centralizzare queste attività, le quali normalmente vengono gestite in modo frammentato. **MealMate** unifica tutto in un'unica esperienza coerente.

1.2 Target Audience

La strategia adottata per definire il target di riferimento è la segmentazione comportamentale. Si definiscono tre principali personas, ovvero tre modelli di utenti in base al motivo per cui utilizzeranno l'applicazione:

- **Studenti e Giovani Professionisti.** Utenti che vivono da soli o con dei coinquilini, con ritmi elevati e risorse economiche da gestire
- **Atleti Amatoriali e Professionisti.** Utenti che tengono al loro stile di vita e che seguono una dieta precisa
- **Famiglie e Genitori Lavoratori.** Utenti che devono gestire l'alimentazione di un gruppo familiare, con poco tempo a disposizione.

Questi gruppi rappresentano i principali target di riferimento dell'applicazione, ma non sono esaustivi.

1.3 Problemi risolti dall'applicazione

L'applicazione intende risolvere diversi problemi

- **Dispersione delle informazioni:** ricette, ingredienti e liste sparse su più supporti.
- **Mancanza di pianificazione:** l'app presenta un'organizzazione anticipata dei pasti grazie alla programmazione settimanale.
- **Inefficienza nella spesa:** acquisto di prodotti già disponibili o mancanza di ingredienti necessari.
- **Gestione migliore della distribuzione economica,** grazie alle liste della spesa.
- **Sprechi alimentari.**

1.4 Principali scenari d'uso

UC- 1 **Aggiunta ricetta**

1. L'utente entra nel menù **Ricette** e clicca sul tasto +.
 2. L'utente inserisce il nome della ricetta, il procedimento, la categoria, il tempo di preparazione, la difficoltà, le porzioni, gli ingredienti ed eventuali note e una foto.
 3. L'utente clicca sul tasto "**salva**" per salvare la ricetta e poterla ricercare.
-

UC- 2 **Aggiunta ricetta ai preferiti**

Precondizioni: l'utente ha inserito almeno una ricetta

1. L'utente entra nel menù **Ricette**
 2. L'utente clicca sul pulsante a forma di cuore presente sulla ricetta per aggiungerla ai preferiti e visualizzarla in una schermata separata.
-

UC- 3 **Dispensa**

1. L'utente entra nel menù **Dispensa** e visualizza la lista di prodotti già in dispensa, ricerca un prodotto o aggiunge un nuovo prodotto cliccando sul tasto verde +.
 2. Cliccando il "tasto verde", l'utente può inserire il nome del prodotto, la categoria, la quantità, l'unità di misura, eventuali note e la data di scadenza, scegliendo da un calendario.
 3. L'utente clicca sul tasto salva per salvare il prodotto nella dispensa
-

UC- 4 **Meal Plan**

1. L'utente entra nel menù **Meal Plan** e visualizza un elenco di pasti pianificati.
2. L'utente può scegliere la settimana, il tipo del pasto e la ricetta per preparare un meal plan.
3. L'utente può cliccare sul tasto "aggiungi al piano" per aggiungere il meal plan all'elenco di pasti pianificati

UC- 5 **Lista Spesa**

1. L'utente entra nel menù **Lista Spesa** e visualizza la lista della spesa, se presente.
2. L'utente scrive il nome di un prodotto, la quantità e l'unità di misura nei rispettivi campi e, cliccando sul tasto verde +, inserisce il prodotto nella lista.

Flusso alternativo

- 2a. L'utente clicca sul tasto Genera dal Meal Plan per generare automaticamente la lista della spesa sulla base dei pasti preparati.
-

UC- 6 **Statistiche**

1. L'utente entra nel menù **Statistiche**
2. Vengono visualizzate diverse informazioni, tra cui le ricette salvate, i prodotti in dispensa, i pasti pianificati, il numero di prodotti nella lista della spesa, il tempo medio di preparazione, i prodotti vicini alla scadenza, i prodotti scaduti. Permette inoltre di visualizzare le ricette preferite.

2 Requisiti

2.1 Funzionalità implementate

Gestione Ricette

- Visualizzazione elenco ricette con ricerca per nome o categoria e filtro per categoria.
- Schermata di dettaglio con immagine, ingredienti, procedimento, tempi e difficoltà.
- Eliminazione ricetta (con rimozione automatica dal meal plan associato).
- Inserimento, modifica ed eliminazione di ricette tramite form completo con aggiunta dinamica di ingredienti.
- Marcatura come preferita e schermata dedicata alle ricette preferite con filtri avanzati.
- Sezione “Consigliate da noi” che suggerisce alcune ricette hardcoded dagli sviluppatori e altre in base agli ingredienti e alle relative quantità disponibili in dispensa.

Gestione Dispensa

- Visualizzazione con ricerca per nome e categoria.
- Raggruppamento automatico in: Prodotti Scaduti, In Scadenza (entro 3 giorni), Altri Prodotti.
- Form completo per aggiunta e modifica con selezione data di scadenza tramite date picker.
- Selezione unità di misura da lista predefinita con opzione personalizzata.
- Eliminazione con bottone dedicato.

Pianificazione Pasti (Meal Plan)

- Vista settimanale con espansione per giorno.
- Aggiunta pasto con selezione giorno, tipologia (Colazione/Pranzo/Cena/Spuntino) e ricetta
- Rimozione di singoli pasti pianificati.
- Contatore pasti per ogni giorno.

Lista della Spesa

- Aggiunta manuale di prodotti con nome, quantità e unità di misura.
 - Generazione automatica dal meal plan (escludendo prodotti già in dispensa).
 - Aggregazione intelligente:
 - prodotti identici con stessa unità vengono sommati.
 - prodotti identici con unità diverse non vengono sommati (l'utente potrebbe volere un determinato prodotto in formati differenti).
 - Spunta come acquistato, selezione multipla ed eliminazione in blocco.
 - Dialog per aggiungere il prodotto acquistato direttamente in dispensa.
-

Statistiche

- Contatori: ricette, prodotti dispensa, pasti pianificati, elementi lista spesa, prodotti in scadenza, prodotti scaduti, tempo medio di preparazione delle ricette.
- Elenco prodotti in scadenza e scaduti con indicatori visivi.
- Accesso rapido alle ricette preferite con contatore.

2.2 Funzionalità considerate e non implementate

- Gestione nutrizionale (calorie, macronutrienti) per limitare il perimetro del progetto.
- Importazione ricette da URL o servizi esterni.
- Integrazione dell'API di Google Calendar per consigliare piatti tipici durante le festività.

2.3 Feature avanzate scelte

- Ricette consigliate: il menù Ricette visualizza in una sezione le ricette consigliate in base alla disponibilità degli ingredienti nella dispensa.
- Generazione automatica Lista Spesa: nel menù Lista Spesa è possibile far generare all'applicazione una lista sulla base del Meal Plan preparato.
- Gestione avanzata delle scadenze con tre livelli di allerta visiva (scaduto, in scadenza, a norma) e ordinamento per data. Sia la sezione Statistiche dell'applicazione sia la sezione Dispensa tengono traccia dei prodotti vicini alla scadenza e dei prodotti scaduti.

2.4 Eventuali limitazioni note

- La gestione delle unità di misura non include conversioni automatiche (es. g → kg): la corrispondenza avviene per stringa esatta.
- Le immagini delle ricette vengono salvate con il percorso assoluto locale, quindi non sono portabili tra dispositivi diversi.

3 Progettazione

3.1 Struttura generale

L'app è strutturata in tre livelli architetturali distinti:

- **Presentazione** (UI Layer): schermate Flutter e widget riutilizzabili.
- **Stato e logica** (State Layer): AppState gestisce lo stato globale e le operazioni sui dati.
- **Persistenza** (Data Layer): DatabaseHelper per SQLite (ricette) e SharedPreferences per gli altri dati.

L'applicazione si suddivide in 5 sezioni principali:

- La sezione **Ricette** permette di cercare, aggiungere o rimuovere una ricetta, aggiungere o rimuovere una ricetta dai preferiti, visualizzare la lista di ricette, la lista di ricette preferite e la lista di ricette consigliate dalla dispensa.
- La sezione **Dispensa** permette di cercare, aggiungere o rimuovere un prodotto e di visualizzare la lista dei prodotti.
- La sezione **Meal Plan** permette di pianificare i pasti di ogni giorno scegliendo una ricetta e di visualizzare i pasti pianificati.
- La sezione **Lista Spesa** permette di aggiungere o rimuovere uno o più elementi dalla lista e di visualizzare la lista della spesa. Permette inoltre di generare una lista automaticamente dal meal plan.
- La sezione **Statistiche** visualizza una serie di statistiche relative alle ricette salvate, ai pasti pianificati, ai prodotti in scadenza o scaduti, alle ricette preferite e calcola il tempo medio di preparazione delle ricette salvate.

3.2 Schermate principali

Schermata	Descrizione
HomeScreen	Schermata di benvenuto con griglia di accesso rapido alle sezioni
RecipesScreen	Elenco ricette con ricerca, filtri per categoria e sezione “Consigliate” a scorrimento orizzontale
RecipeDetailScreen	Dettaglio ricetta con header immagine, informazioni aggiuntive, ingredienti e procedimento
RecipeFormScreen	Form di inserimento/modifica con righe ingredienti e selezione immagine dalla galleria
FavoriteRecipesScreen	Lista preferiti con filtri avanzati per categoria, difficoltà e tempo di preparazione
PantryScreen	Dispensa con raggruppamento automatico per stato scadenza e ricerca full-text
PantryFormScreen	Form aggiunta/modifica prodotto con date picker e dropdown unità di misura
MealPlanScreen	Pianificazione settimanale con vista a fisarmonica per giorno
ShoppingListScreen	Lista della spesa con aggiunta manuale, generazione automatica e dialog di trasferimento in dispensa
StatsScreen	Statistiche e riepilogo con indicatori visivi per prodotti in scadenza e scaduti

Tabella 1: Schermate principali dell'applicazione

3.3 Wireframe

In questa sezione vengono presentati i **wireframe** preliminari progettati per lo sviluppo dell'applicazione mobile.

Questi schemi, affinati attraverso diverse iterazioni, hanno fornito una fondamentale linea guida visiva e strutturale, fungendo da base per l'implementazione dell'interfaccia utente (UI) e della logica applicativa di ogni schermata.



Figura 1: Wireframe della home page

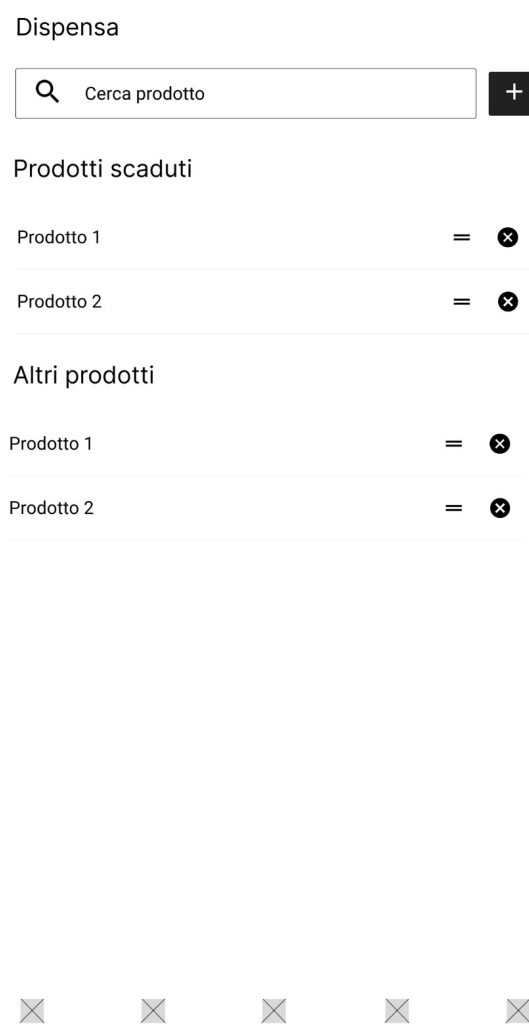


Figura 2: Wireframe di Dispensa

Lista Spesa

Quantità	Unità ▾	Prodotto	+
GENERA DAL MEAL PLAN			
SELEZIONA TUTTI		ELIMINA SELEZIONE	
✕	Ingrediente 1		☐
✕	Ingrediente 2		☒



Figura 3: Wireframe di Lista-spesa

Meal Plan

Settimana 1
Aggiungi un pasto
Giorno ▾
Pasto ▾
Ricetta ▾
+ AGGIUNGI AL PIANO
La tua settimana
Lunedì ▾
Martedì ▾
Mercoledì ▾
Giovedì ▾
Venerdì ▾
Sabato ▾
Domenica ▾



Figura 4: Wireframe di Meal-Plean

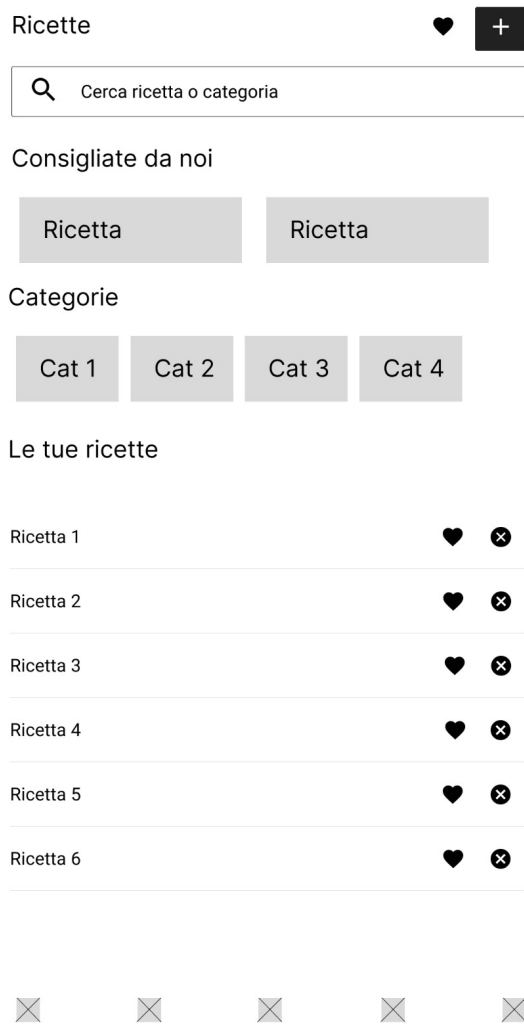


Figura 5: Wireframe di Ricette

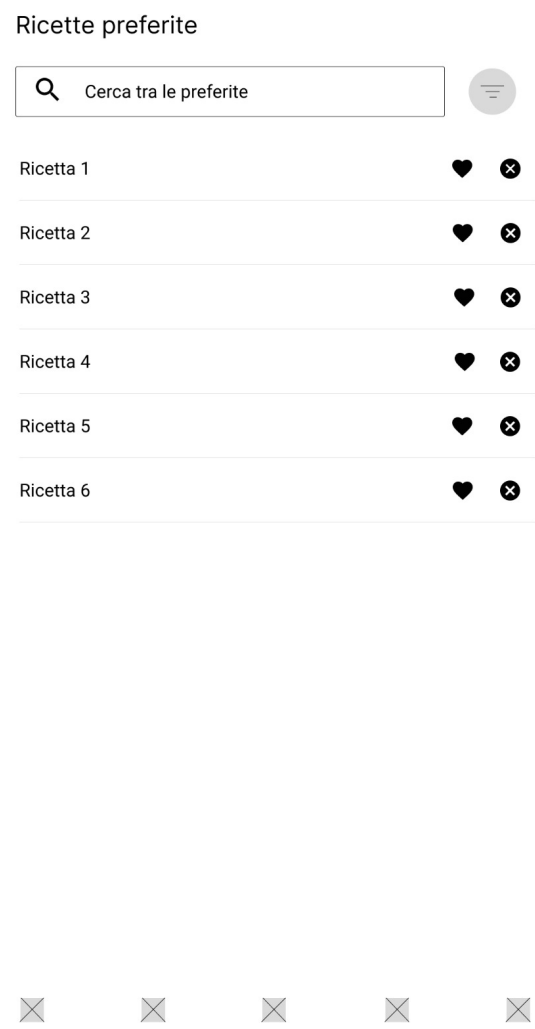


Figura 6: Wireframe di Ricette-Preferite

Statistiche

Ricette salvate	1
Prodotti in dispensa	2
Pasti pianificati	5
Elementi in lista della spesa	2
Tempo di preparazione	15 min
Prodotti scaduti	1

Prodotti in scadenza

Prodotti scaduti

! Ingrediente 1

! Ingrediente 2

Ricette preferite



Figura 7: Wireframe della pagina Statistiche

3.4 Organizzazione dei Dati

Dati dell'app sono organizzati in **5 entità principali**:

Entità	Persistenza	Campi principali
Recipe	SQLite	id, name, description, category, preparationTime, difficulty, portions, notes, imagePath, isRecommended, isFavorite
RecipeIngredient	SQLite	id, recipeId (FK), name, quantity, unit
PantryItem	SharedPreferences	id, name, category, quantity, unit, expiryDate, notes
MealPlanItem	SharedPreferences	id, weekStart, day, mealType, recipeId
ShoppingItem	SharedPreferences	id, name, quantity, unit, purchased

Tabella 2: Tabella riassuntiva dell'organizzazione dei dati

4 Scelte tecnologiche

4.1 Framework utilizzato

Flutter è stato scelto per la sua capacità di produrre app native ad alte prestazioni su iOS e Android a partire da un'unica codebase.

Il sistema di widget dichiarativi permette di costruire interfacce complesse in modo mantenibile.

Dart, il linguaggio sottostante, offre tipizzazione forte, null safety e asincronia tramite `async/await`, caratteristiche essenziali per un'app che gestisce I/O su database e file system.

4.2 Librerie principali

Libreria	Utilizzo
provider	Gestione dello stato globale tramite ChangeNotifier + Consumer pattern
sqflite	Database SQLite locale per ricette e ingredienti con relazioni e foreign key
shared_preferences	Persistenza key-value per dispensa, meal plan e lista spesa in formato JSON
path / path_provider	Risoluzione del percorso del database SQLite sul filesystem del dispositivo
image_picker	Selezione immagine dalla galleria del dispositivo per le ricette
intl	Formattazione delle date in formato dd/MM/yyyy nella schermata dispensa

Tabella 3: Schermate principali dell'applicazione

5 Implementazione

5.1 Widget principali

L'applicazione utilizza diversi widget e componenti personalizzati per implementare le funzionalità principali

MainScreen è un *StatefulWidget* che ospita le cinque sezioni principali tramite *IndexedStack*, che mantiene lo stato di ogni schermata attiva anche quando non visibile, evitando ricreazioni costose al cambio tab.

La navigazione tra tab avviene tramite *BottomNavigationBar*.

- **IngredientRow**

Classe di supporto utilizzata nel form delle ricette per gestire dinamicamente gli ingredienti.

Ogni riga contiene i controller per nome, quantità e unità di misura e può essere aggiunta o rimossa dall'utente durante la compilazione della ricetta.

- **Recipe Card**

Componente utilizzato nella schermata **Ricette** per visualizzare sinteticamente le informazioni principali di una ricetta, tra cui nome, categoria, immagine e stato preferito.

- **ChoiceChip** utilizzato in *RecipeFormScreen* per la selezione della difficoltà (Facile, Media, Difficile) e in *FavoriteRecipesScreen* per i filtri avanzati nel bottom sheet.

Permette una selezione visiva immediata tra opzioni mutualmente esclusive.

- **AlertDialog** riutilizzato in più contesti con comportamenti diversi:

- conferma eliminazione ricetta in *RecipeDetailScreen*.
- avviso ingredienti mancanti in *MealPlanScreen*.
- trasferimento prodotto in dispensa in *ShoppingListScreen* e modifica pasto in *MealPlanScreen*.

In tutti i casi con **StatefulBuilder** dove necessario per aggiornare lo stato interno.

- **ModalBottomSheet** con *showDragHandle* in *FavoriteRecipesScreen* per il pannello filtri, che utilizza variabili temporanee per non applicare i filtri finché l'utente non preme "Applica", con possibilità di reset.

5.2 Organizzazione del codice

Il progetto segue una struttura a layer chiaramente separati:

- `lib/models/` → Classi dati pure (`Recipe`, `RecipeIngredient`, `PantryItem`, `MealPlanItem`, `ShoppingItem`) con metodi `toJson/fromJson` e `toMap/fromMap` per la serializzazione.
- `lib/providers/` → `AppState`, unico provider di stato che espone i dati e i metodi di business logic a tutte le schermate.
- `lib/database/` → `DatabaseHelper`, singleton che gestisce la connessione SQLite e tutte le operazioni sulle ricette.
- `lib/screens/` → Tutte le schermate dell'app.

5.3 Gestione della Navigazione

La `HomeScreen` serve come punto di ingresso con accesso rapido a tutte le sezioni. Dalla `HomeScreen`, il tap su una voce apre `MainScreen` con il tab corrispondente pre-selezionato tramite il parametro `initialIndex`.

La navigazione sfrutta due meccanismi:

- `Navigator.push()` con `MaterialPageRoute` per le transizioni verso schermate secondarie (form, dettagli, preferite).
Il ritorno avviene tramite `Navigator.pop()` o il tasto back del dispositivo.
- `IndexedStack` nella `MainScreen` per le cinque tab principali (Ricette, Dispensa, Meal Plan, Lista Spesa, Statistiche).
Questo pattern mantiene lo stato di ogni schermata (scroll position, dati caricati) anche quando non è visibile, evitando ricreazioni costose.

Il passaggio di dati tra schermate avviene in tre modi:

1. tramite costruttore (es. `recipeId` a `RecipeDetailScreen`).
2. tramite Provider (accesso diretto ad `AppState` da qualsiasi schermata).
3. tramite `await` su `Navigator.push` per aspettare il risultato di un form prima di aggiornare la UI.

5.4 Gestione dello Stato

AppState estende ChangeNotifier e viene iniettato nella widget tree tramite ChangeNotifierProvider al livello più alto dell'app.

Ogni schermata accede allo stato tramite Provider.of<AppState>(context) o Consumer<AppState>.

Le operazioni che modificano lo stato seguono sempre il pattern:

memoria → persistenza asincrona su disco → notifyListeners() per aggiornare la UI. Questo garantisce che la UI sia sempre sincronizzata con i dati persistiti.

Per le ricette, che risiedono su SQLite, il pattern è leggermente diverso:

l'operazione sul database viene completata prima di ricaricare la lista aggiornata tramite loadRecipesFromDatabase(), evitando stati inconsistenti.

5.5 Gestione della Persistenza

La scelta di usare **due sistemi** di persistenza distinti è motivata dalla natura dei dati:

- **SQLite** per le ricette e i loro ingredienti: le ricette e i loro ingredienti sono salvati su SQLite con una relazione uno-a-molti(ON DELETE CASCADE). SQLite gestisce nativamente join, query filtrate e integrità referenziale con le foreign key e le operazioni CASCADE. La versione del database è predisposta tramite _upgradeDB per supportare migrazioni future.
- **SharedPreferences** per gli altri dati: dispensa, meal plan e lista spesa sono liste piatte senza relazioni tra loro. La serializzazione JSON su SharedPreferences è più semplice da implementare e sufficientemente performante per le dimensioni tipiche di questi dataset.

Entrambe le soluzioni garantiscono la persistenza dei dati tra sessioni dell'app senza richiedere connettività o backend remoti.

5.6 Parti di codice significative o complesse

Generazione Automatica della Lista della Spesa

Il metodo `generateShoppingListFromMealPlan()` in **AppState** implementa la logica più complessa dell'app.

Per ogni pasto nel meal plan, recupera la ricetta associata e itera sugli ingredienti.

Prima di aggiungere un ingrediente alla lista, il sistema confronta gli ingredienti richiesti dalle ricette pianificate con quelli presenti in dispensa, considerando nome, unità di misura e quantità disponibile.

Se la quantità presente è insufficiente, viene aggiunta alla lista della spesa soltanto la quantità mancante.

Mentre se l'ingrediente è già nella lista della spesa con la stessa unità, somma le quantità invece di duplicare la voce.

Questo approccio evita ridondanze e produce una lista spesa ottimizzata e pulita.

Database SQLite con Relazione Uno-a-Molti

La gestione del database in **DatabaseHelper** affronta il problema classico delle relazioni in SQLite.

L'operazione `"getRecipes()"` esegue prima una query sulla tabella `recipes`, poi per ogni ricetta una query secondaria su `recipe_ingredients` filtrata per `recipeId`.

Questa strategia è accettabile per le dimensioni tipiche di un ricettario personale e mantiene il codice semplice e leggibile.

Form Ingredienti Dinamico

RecipeFormScreen implementa un sistema di righe ingredienti completamente dinamico tramite la classe di supporto **IngredientRow**, che incapsula i tre **TextEditingController** di ogni riga (nome, quantità, unità).

Il metodo `dispose()` di ogni **IngredientRow** viene richiamato correttamente quando la riga viene rimossa o quando il form viene chiuso, prevenendo memory leak.

Suggerimento Ricette

Il metodo `suggestedRecipes()` in **AppState** restituisce le ricette che hanno almeno un ingrediente presente in dispensa oppure che sono state marcate come raccomandate (`isRecommended`).

Le ricette raccomandate vengono mostrate con priorità e il numero massimo di suggerimenti visualizzati è limitato a cinque.

Il confronto avviene per nome normalizzato (`toLowerCase().trim()`) per gestire variazioni di maiuscolo/minuscolo e spazi.

Trasferimento Lista Spesa → Dispensa

La funzione `showAddToPantryDialog()` in `ShoppingListScreen` implementa un `dialog` contestuale che appare quando l'utente preme il tasto elimina su un elemento della lista.

Il `dialog` propone di aggiungere il prodotto in dispensa con categoria, note e data di scadenza, oppure di eliminarlo semplicemente.

Questo aumenta la semplicità con cui è possibile tenere la dispensa aggiornata dopo la spesa.

6 Conclusione

Il progetto ha consentito di applicare i principali concetti affrontati durante il corso di **Mobile Programming**, tra cui gestione dello stato, navigazione, persistenza locale, progettazione dell'interfaccia utente e organizzazione del codice.

L'applicazione sviluppata risulta completa, estendibile e utilizzabile in un contesto reale per la pianificazione dei pasti e la gestione della dispensa domestica.